Securitize Bridge ACL Ext Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

JesJupyter

Ctrus

July 30, 2026

Contents

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Securitize's Solana bridge moves regulated real-world-asset (RWA) tokens between EVM chains and Solana, gating transfers on Securitize's on-chain identity and whitelist programs. This delta adds the `TokenConfig::Spl` token path, in which a token's mint authority is held by an external Securitize ACL program and its freeze authority by the whitelist program, alongside the pre-existing `TokenConfig::Ds` path (mint authority held by the asset controller), together with companion changes to the `spl_token_whitelist` program.

Inbound EVM-to-Solana transfers arrive as Wormhole VAAs: the SPL path checks an `InvestorRegistry` PDA derived on-chain from the destination mint and wallet before minting through the ACL program, while the DS path validates the recipient through the Securitize identity-registry and RBAC programs. Outbound, SPL tokens are burned directly under the user's own signature via Token-2022 `burn_checked`, whereas DS tokens are revoked through an RBAC CPI. Additional changes to the `spl_token_whitelist` program allow for the creation and deletion of `InvestorRegistry` PDAs, which are used by the bridge to validate investor eligibility on-chain.

5 Audit Scope

The audit scope was limited to:

```
// new files
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/admin/update_spl_token_registry_program
↪  _id.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/bridge/bridge_spl_tokens.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/bridge/execute_vaa_v1_spl.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/resolver/ds.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/resolver/mod.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/resolver/result_pda.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/resolver/spl.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/resolver/vaa_body.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/state/investor_registry.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/bridge_send/checks.rs
```

```
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/bridge_send/wormhole_cpi.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/locked_tokens/policy_engine.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/parse_rbac_accounts.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/validate_and_decode_vaa.rs
bc-solana-bridge-sc/programs/spl-ac-program-interface/src/lib.rs

// some changes
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/admin/initialize.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/bridge/bridge_ds_tokens.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/bridge/execute_vaa_v1.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/state/config.rs

// very minor changes
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/admin/set_emitter_address.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/admin/update_authorized_user_role.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/bridge_send/executor_cpi.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/locked_tokens/mod.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/locked_tokens/tracker.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/events.rs
bc-solana-bridge-sc/programs/securitize_bridge/src/lib.rs
bc-solana-bridge-sc/programs/securitize_usdc_bridge/src/instructions/admin/initialize.rs
bc-solana-bridge-sc/programs/securitize_usdc_bridge/src/instructions/admin/set_cctp_domain.rs
bc-solana-bridge-sc/programs/securitize_usdc_bridge/src/instructions/admin/update_min_finality_threshol
↪  d.rs
bc-solana-bridge-sc/programs/securitize_usdc_bridge/src/instructions/bridge/send_usdc_cross_chain_depos
↪  it.rs

// new files
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/instructions/admin/create_investor_registry.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/instructions/admin/delete_investor_registry.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/states/investor_registry.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/utils/require_freeze_authority.rs

// some changes
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/instructions/whitelist.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/lib.rs

// minor changes
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/constants.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/events.rs

// very minor changes
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/errors.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/instructions/admin/mod.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/states/mod.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/src/utils/mod.rs
bc-solana-whitelist-sc/programs/spl-token-whitelist/Cargo.toml
```

6 Executive Summary

Over the course of 6 days, the Cyfrin team conducted an audit on the Bridge ACL Ext code provided by Securitize. The findings consist of 4 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

- Both bridge programs are upgradeable through their program-data upgrade authority; whoever holds that authority can replace any program logic, including the whitelist and mint checks, so end-users ultimately trust the upgrade-authority custodian. The custody arrangement (single key, multisig, or immutable) is a deployment-time property the client should confirm and publish.

- Each bridge program is controlled by a per-program `owner` account with broad administrative power over its per-mint configuration. Within the audited delta this includes rotating the trusted Wormhole emitter address, rotating the SPL token-registry program id, and setting the authorized user role on `securitize_bridge`, and configuring CCTP destination domains and the finality threshold on `securitize_usdc_bridge`. Each program additionally exposes further owner-gated administrative instructions outside this delta scope (for example pausing, fee and executor-fee configuration, bridge-caller allowlist management, and withdrawing the USDC config PDA's SOL reserve); end-users should treat the owner as fully trusted over each bridge's configuration and the funds it custodies.
- Rotating the trusted Wormhole emitter address strands in-flight inbound transfers: VAAs already posted by the previous emitter, whose source tokens are already burned, fail validation until the owner restores the prior emitter, and no grace window covers the transition.
- Rotating the SPL token-registry program id strands already-whitelisted investors' in-flight inbound transfers, because their `InvestorRegistry` PDAs remain owned by the previous program and fail the post-rotation owner check until they are re-whitelisted; no migration path or two-phase window exists.
- `delete_investor_registry` strands in-flight inbound transfers for the affected `(mint, wallet)` pair. Once the `InvestorRegistry` account is closed, inbound VAAs targeting that recipient fail the on-chain registry check until a new record is created.
- Closing an `InvestorRegistry` through `delete_investor_registry` also refunds its rent to a caller-chosen receiver, defaulting to the deleting admin or freeze-authority signer. For records provisioned through the public whitelist path (where the investor wallet funded the rent), this redirects the rent-exempt deposit away from the original payer.
- Whitelisting and de-whitelisting of investors is restricted to the whitelist program admin or the mint's freeze authority, both of which are trusted Securitize-operated principals.

6.2 Defensive Improvements

- CCTP `max_fee` and `min_finality_threshold` can be configured into an incoherent combination (`bc-solana-bridge-sc/programs/securitize_usdc_bridge/src/instructions/bridge/send_usdc_cross_chain_deposit.rs:283-284`). The two values are written by independent admin setters with no cross-check, so lowering the finality threshold below 2000 (Circle's Fast-transfer regime) while `max_fee` is zero makes `deposit_for_burn` revert on every deposit until an admin reconciles them. Validate the pairing whenever either value is set.
- The USDC deposit path enforces an EVM-shaped recipient regardless of the configured CCTP destination (`bc-solana-bridge-sc/programs/securitize_usdc_bridge/src/instructions/bridge/send_usdc_cross_chain_deposit.rs:177-180`). It always requires the 32-byte recipient to be a left-padded 20-byte EVM address, so registering any non-EVM Circle domain (such as Solana, Aptos, or Sui) would reject legitimate native recipients. Gate the EVM-shape check on the destination domain, or restrict domain registration to EVM lanes.

Summary

Project Name	Bridge ACL Ext
Repository	bc-solana-bridge-sc
Commit	1e90b616569c...
Fix Commit	033b8e95c481...
Repository 2	bc-solana-whitelist-sc
Commit	bc886323f93b...
Fix Commit	d6d19fd83837...
Audit Timeline	June 10th - June 17th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	4
Informational	7
Gas Optimizations	1
Total Issues	12

Summary of Findings

[L-1] `send_usdc_cross_chain_deposit` unconditionally reimburses the executor fee from the protocol `config` PDA with an unconstrained `payer` and `executor_payee`	Resolved
[L-2] `InvestorRegistry::load` skips the Anchor discriminator check before Borsh-deserializing the whitelist account body	Resolved
[L-3] `securitize_bridge::initialize` applies no validation to the SPL `acl_program_id`, permanently bricking an instance from a zero or garbage value	Resolved
[L-4] De-whitelisting closes `InvestorRegistry` but does not re-freeze the owner's token accounts	Acknowledged
[I-1] `securitize_bridge` inbound SPL resolver hardcodes the Token-2022 program for ATA derivation, breaking inbound delivery for a classic-SPL-Token mint configured as `TokenConfig::Spl`	Resolved
[I-2] Inconsistent `investor_id` length limits across layers (64 vs 256) and an undocumented 32-byte PDA-seed constraint on the DS path	Resolved
[I-3] Missing upfront validation during initialization	Acknowledged
[I-4] Outbound DS/SPL bridge handlers do not validate EVM recipient address format	Resolved

[I-5] Empty `investor_id` can be persisted in `InvestorRegistry` and propagated through the SPL bridge	Resolved
[I-6] `tracker_account` binding is not validated upfront in `bridge_ds_tokens`	Resolved
[I-7] Whitelist pause does not stop investor registry create/delete paths	Acknowledged
[G-1] Redundant heap clone of the up-to-1024-byte VAA payload on every inbound bridge transaction	Resolved

7 Findings

7.1 Low Risk

7.1.1 `send_usdc_cross_chain_deposit` unconditionally reimburses the executor fee from the protocol `config` PDA with an unconstrained `payer` and `executor_payee`

Description: On the USDC bridge path, the executor relay fee is sourced from the protocol's own `config` PDA rather than from the user, and the reimbursement is unconditional. `securitize_usdc_bridge::send_usdc_cross_chain_deposit` runs three steps in sequence (`send_usdc_cross_chain_deposit.rs:200-210`): `validate_payer_lamports_for_executor`, then `invoke_request_execution_cpi`, then `reimburse_executor_fee`. The CPI issues a `request_for_execution` call whose only fund movement is a system transfer of `exec_amount` lamports from `payer` to `executor_payee` (`send_usdc_cross_chain_deposit.rs:356-374`). Immediately after, `reimburse_executor_fee` does `config.sub_lamports(exec_amount); payer.add_lamports(exec_amount)` (`send_usdc_cross_chain_deposit.rs:337-338`), crediting `exec_amount` lamports from the protocol `config` PDA back to `payer`.

Two missing constraints turn this reimbursement into a drain. First, `executor_payee` is a bare `UncheckedAccount` (`send_usdc_cross_chain_deposit.rs:155-157`) with no validation that it is the legitimate Wormhole executor payee, and there is no check that `executor_payee != payer`. Second, `payer` is an unconstrained `Signer` (`send_usdc_cross_chain_deposit.rs:18-21`); the `bridge_caller` gate covers only `caller` (the USDC owner), not `payer`. A caller can therefore set `payer == executor_payee` so the executor CPI is a net-zero self-transfer, while the unconditional reimbursement still pays `payer` `exec_amount` out of the `config` PDA. The reimbursement never verifies that a relay was actually purchased, that `exec_amount` matches the off-chain signed quote, or that the lamports left `payer`'s control. By contrast, the other two executor-fee call sites (`securitize_bridge::bridge_ds_tokens, bridge_spl_tokens`) have the user's `payer` pay `exec_amount` directly with no reimbursement from protocol funds, so they do not leak protocol value.

The Wormhole Executor being a trusted, out-of-scope component does not block this. The Executor (program `execXUrAsMnqMmTHj5m7N1YQgsDz3cwGLYCYyuDRciV`, Wormhole `example-messaging-executor`) is designed to stay lightweight and performs no on-chain signature verification of the signed quote, leaving quote verification to the submitting client; its `request_for_execution` only checks that the passed `payee` account equals bytes `[24..56]` of the quote, that the quote's source/destination chain ids match, and that its expiry is in the future, then does a plain `system_program` transfer of `amount` from `payer` to `payee`. The Executor's README states this explicitly: "this contract MUST NOT verify the signature on the Quote ... The Quote SHOULD be verified by the submitting client code." Here the bridge is that client, and `send_usdc_cross_chain_deposit` forwards the caller-supplied `signed_quote_bytes` to the Executor unmodified and never verifies them. An attacker therefore forges a well-formed (unsigned) quote naming themselves as `payee` with matching chain ids and a far-future expiry, sets `payer == executor_payee == attacker`, and the real Executor accepts it and performs a net-zero self-transfer; the bridge then reimburses the attacker from `config`. The accompanying PoC constructs exactly this forged quote.

Files:

- `securitize_usdc_bridge::send_usdc_cross_chain_deposit` - `bc-solana-bridge-sc/programs/securitize_usdc_bridge/src/instructions/bridge/send_usdc_cross_chain_deposit.rs:18-21,155-157,200-210,329-375`

Impact: Direct theft of the protocol-owned `config` PDA's SOL. Each call nets the caller up to `config.max_executor_fee` lamports from the `config` PDA, since `exec_amount` is bounded only by `require_gte!(config.max_executor_fee, exec_amount)` (`send_usdc_cross_chain_deposit.rs:194-198`). The call is repeatable until `validate_executor_fee_balance` (`send_usdc_cross_chain_deposit.rs:299-303`) can no longer satisfy `config.lamports >= rent_min + exec_amount`, i.e. until the entire SOL surplus the protocol holds to sponsor relays is drained down to the rent-exempt minimum. No price movement, admin error, or third-party donation is required, and because the CCTP burn still executes, each draining call looks like an ordinary bridge transfer. Even a benign whitelisted caller acting in self-interest receives a free `exec_amount` subsidy that never funds a relayer, so the relay the protocol paid for is never performed while the protocol still bears the cost. The Wormhole Executor's trusted, out-of-scope status does not mitigate this: by design it never verifies the quote signature, so an attacker-forged quote naming themselves as `payee` is accepted and no real relay is purchased. A prior Securitize Solana bridge audit already treated depletion of this same `config` PDA SOL

reserve through executor fees as a valid issue.

Proof of Concept: The following Anchor test was added to the existing `#sendUsdcCrossChainDeposit` test suite (which loads the mock CCTP and mock executor programs via the test-validator genesis) and passes (`1 passing`). A single allowlisted caller sets `payer == executor_payee`; after one `send_usdc_cross_chain_deposit` the protocol `config` PDA is debited by exactly `exec_amount` while the attacker's balance rises, and no relay is ever delivered:

```
// Add this test to `tests/specs/securitize-usdc-bridge.spec.ts` inside the
// `describe("#sendUsdcCrossChainDeposit", ...)` block (it reuses that block's
// fixtures: an initialized USDC bridge config, a registered bridge_caller for
// `ownerKp`, and `realCallerTokenAccount` owned by `ownerKp`).
//
// Run with: anchor test --skip-build
// (Node 24; the mock CCTP token-messenger/message-transmitter and mock executor
// programs are loaded from tests/programs/ via Anchor.toml [[test.genesis]].)
//
// The signed quote is FORGED. The real Wormhole Executor (program
// execXUrAsMnqMmTHj5m7N1YQgsDz3cwGLYCYyuDRciV) performs NO on-chain signature
// verification of the quote and request_for_execution only checks
// payee == signed_quote_bytes[24..56], the src/dst chains at [56..60], and the
// expiry at [60..68] - all attacker-controlled. So a forged, well-formed quote
// naming the attacker as payee passes the real executor exactly as it passes the
// mock here. The bridge forwards this caller-supplied quote to the executor
// unmodified and never verifies it.
//
// Result: 1 passing. The config PDA is debited by exactly exec_amount and the
// attacker's balance increases, with no relay ever delivered.
it("PoC: allowlisted caller drains the config PDA SOL reserve via the unconditional executor-fee
↪  reimbursement", async () => {
  const conn = provider.connection;

  // The protocol funds the config PDA with a relay-sponsorship SOL float (a
  // precondition for ANY sponsored relay; validate_executor_fee_balance requires
  // config.lamports >= rent_min + exec_amount). Not attacker-controlled.
  const configPda = usdcBridgeConfigPda(realUsdcMint);
  await airdropSol(conn, configPda, 5);

  // Attacker-controlled signer A is used as BOTH `payer` and `executor_payee`.
  // `caller` stays the legitimately-allowlisted wallet (bridge_caller PDA registered
  // in `before`), and its USDC account funds the burn - the deposit looks ordinary.
  const attacker = Keypair.generate();
  await airdropSol(conn, attacker.publicKey, 3); // > exec_amount, for the pre-CPI payer-balance check

  // exec_amount is bounded only by config.max_executor_fee (10 SOL in tests).
  const execAmount = 1 * LAMPORTS_PER_SOL;

  const depositAmount = 500 * ONE_USDC;
  await mintUsdcTo(
    conn,
    ownerKp,
    realUsdcMint,
    realCallerTokenAccount,
    realMintAuthority,
    depositAmount
  );

  const recipient = randomEvmRecipient();

  // Forge a well-formed (unsigned) Executor quote: payee = attacker at [24..56],
  // src_chain = Solana (1) at [56..58], dst_chain = ETHEREUM_CHAIN_ID at [58..60],
  // expiry = u64::MAX at [60..68]. The real executor accepts this (no sig check);
  // the mock ignores it. Layout per execXUrAsMnqMmTHj5m7N1YQgsDz3cwGLYCYyuDRciV.
```

```
  const forgedQuote = Buffer.alloc(68);
  attacker.publicKey.toBuffer().copy(forgedQuote, 24);
  forgedQuote.writeUInt16BE(1, 56);
  forgedQuote.writeUInt16BE(ETHEREUM_CHAIN_ID, 58);
  forgedQuote.writeBigUInt64BE(BigInt("18446744073709551615"), 60);

  const configBefore = await conn.getBalance(configPda);
  const attackerBefore = await conn.getBalance(attacker.publicKey);

  const result = await SecuritizeUsdcBridgeClient.instructions.sendUsdc(ctx.bridge, {
    targetChain: ETHEREUM_CHAIN_ID,
    recipient,
    amount: depositAmount,
    execAmount,
    signedQuoteBytes: forgedQuote,
    usdcMint: realUsdcMint,
    callerUsdcAccount: realCallerTokenAccount, // owned by the allowlisted caller
    cctpDomain: ETHEREUM_CCTP_DOMAIN,
    executorAccounts: {
      executorProgram: EXECUTOR_PROGRAM_ID,
      payee: attacker.publicKey, // executor_payee == payer => executor CPI nets zero
    },
    ownerKp: attacker, // signs as `payer` (decoupled from caller)
    callerKp: ownerKp, // signs as `caller` (the allowlisted bridge_caller)
  });

  const tx = new Transaction().add(...result.instructions);
  // Make the allowlisted caller the tx fee payer so the attacker's balance delta
  // reflects only the on-chain reimbursement logic, not transaction fees.
  tx.feePayer = ownerKp.publicKey;
  await provider.sendAndConfirm!(tx, [attacker, ...result.signers]);

  const configAfter = await conn.getBalance(configPda);
  const attackerAfter = await conn.getBalance(attacker.publicKey);

  // The protocol-owned config PDA is drained by exactly exec_amount...
  expect(configBefore - configAfter).to.equal(execAmount);
  // ...and the attacker pockets it (net positive even after paying the CCTP
  // message-account rent), with no relay ever delivered.
  expect(attackerAfter).to.be.greaterThan(attackerBefore);
});
```

Textual Step-by-Step Proof:

1. The owner registers a `bridge_caller` PDA for caller `K` (normal operational setup). The `config` PDA holds SOL above its rent-exempt minimum, as it must for any sponsored relay, since `validate_executor_fee_balance` requires `config.lamports >= rent_min + exec_amount`.
2. The attacker controls a whitelisted caller `K` and an arbitrary signer `A`. They build a `send_usdc_cross_chain_deposit` transaction with `caller = K`, `payer = A`, `executor_payee = A` (the same account as `payer`), `exec_amount = config.max_executor_fee`, and a legitimate `amount` of USDC from `K`'s token account satisfying `amount > config.max_fee`.
3. The handler passes all checks: the `bridge_caller` PDA validates `K`, `caller_usdc_account` is owned by `K`, `validate_executor_fee_balance` passes because the `config` PDA is funded, and `validate_payer_lamports_for_executor` passes because `A` holds at least `exec_amount`.
4. `invoke_deposit_for_burn` executes the CCTP `deposit_for_burn`, so the USDC bridges normally.
5. `invoke_request_execution_cpi` issues the executor CPI, whose system transfer is `A -> A` of `exec_amount` lamports - a net-zero self-transfer that leaves `A`'s balance unchanged.
6. `reimburse_executor_fee` then runs unconditionally: `config.sub_lamports(exec_amount); A.add_lamports(exec_amount)`. `A` gains `exec_amount` lamports drawn from the protocol `config` PDA.

7. Net result per call: the `config` PDA loses `exec_amount` (up to `max_executor_fee`) and `A` pockets it, while the transaction otherwise looks like a normal bridge transfer. The attacker repeats the call until `config.lamports` falls to the rent-exempt minimum, draining the protocol's entire relay-sponsorship SOL float.

Recommended Mitigation: Stop reimbursing the executor fee from protocol funds based on a caller-supplied `exec_amount`. Either:

- (a) Have the user's `payer` pay the executor with no reimbursement, matching the `bridge_ds_tokens`, `bridge_spl_tokens` paths; remove `reimburse_executor_fee` entirely; or
- (b) If the protocol must sponsor relays, do not make the data-bearing `config` PDA the Executor CPI's `payer`: the Executor moves funds with a `system_program` transfer, which can only debit a system-owned account, whereas `config` is a program-owned data account (`Box<Account<UsdcBridgeConfig>>`) whose lamports can only be moved by direct `sub_lamports`/`add_lamports`. Instead either (i) keep the user's `payer` fronting the fee and reimbursing it from `config` by direct lamport debit, or (ii) front the fee from a dedicated system-owned funding PDA that signs the Executor CPI via seeds. In both cases the payee constraint alone is not sufficient, because the Wormhole Executor does not verify the signed quote on-chain: gate the `config`/funding-PDA spend on an in-program verification of the signed quote - check the quoter signature against an approved quoter key (or use a canonical on-chain quote source) and bind the funded `exec_amount`, `executor_payee`, source/destination chains, and expiry to that verified quote - and require `executor_payee != payer` so funds are only ever spent on an authentic, relayed request.

In either case, do not move `config` PDA funds - whether by reimbursing `payer` or by paying the executor directly - without a verified binding between the spent amount and a real, authenticated outbound relay payment.

Securitize: Fixed in commit 6893436d. The protocol no longer sponsors the executor fee: reimburse_executor_fee (and the config.sub_lamports / payer.add_lamports reimbursement) was removed entirely, so config PDA funds are never moved on this path. The executor fee is now paid directly by the user's payer via the executor CPI's payer and payee system transfer, with no reimbursement — matching the bridge_ds_tokens and bridge_spl_tokens paths. This eliminates the drain: a net-zero self-transfer no longer yields any payout from protocol funds.

Cyfrin: Verified.

7.1.2 `InvestorRegistry::load` skips the Anchor discriminator check before Borsh-deserializing the whitelist account body

Description: On the inbound SPL path the bridge reads the external whitelist record through `InvestorRegistry::load`. The loader borrows the account data, requires only that `data.len() >= 8`, then skips the first 8 bytes and Borsh-deserializes the `mint`, `wallet`, `investor_id`, and `bump` fields from the remainder. It never compares those leading 8 bytes against the Anchor discriminator `sha256("account:InvestorRegistry")[..8]`, so the account *type* is never verified at deserialization time:

```
let mut slice = &data[ANCHOR_DISCRIMINATOR_LEN..];
let view = Self::deserialize(&mut slice)
    .map_err(|_| error!(BridgeError::InvalidInvestorRegistry))?;
```

This is currently not exploitable. In `execute_vaa_v1_spl` and `bridge_spl_tokens` the `investor_registry` account is an `UncheckedAccount` pinned by Anchor `seeds = [INVESTOR_REGISTRY_SEED_PREFIX, asset_mint, wallet]`, `seeds::program = <configured registry program>`, and `owner = <configured registry program>`. The registry program creates only `InvestorRegistry`-typed accounts at that seed prefix, and the `owner` constraint already rejects system-owned, zero-lamport, or closed accounts. So a type-confusion or zeroed-account read is not reachable today; the address and owner pins do the work the discriminator check would otherwise do. . This is the only registry-read step that does not enforce the account type by its discriminator.

Files:

- `securitize_bridge::InvestorRegistry::load` - `bc-solana-bridge-sc/programs/securitize_bridge/src/state/investor_registry.rs:38-57`

Impact: No impact under the current single-account-type registry program and the seeds/owner pinning - the missing check is defense-in-depth, not a live vulnerability. It becomes a real type-confusion vector only if the

registry program is later extended to own a second account type derivable at a colliding seed prefix, or if the bridge's configured registry program id is repointed to a program with a different account layout: in either case the body would be parsed as an `InvestorRegistry` from raw bytes with no discriminator gate, and a non-registry account could satisfy the whitelist check. The fix is cheap and removes the latent exposure while restoring parity with the DS path.

Recommended Mitigation: In `load`, before deserializing the body, assert the leading 8 bytes equal the expected Anchor discriminator for the `InvestorRegistry` account, mirroring the existing DS-path check in `load_imr_investor`. Compute the discriminator as `sha256("account:InvestorRegistry")[..8]` (or reference the whitelist program's published constant) and `require!` equality before reading `data[8..]`, returning `BridgeError::InvalidInvestorRegistry` on mismatch.

Securitize: Fixed in commit 394be0f. InvestorRegistry::load now asserts the leading 8 bytes equal the Anchor discriminator sha256("account:InvestorRegistry")[..8] (require!(data.starts_with(&INVESTOR_REGISTRY_DISCRIMINATOR), InvalidInvestorRegistry)) before Borsh-deserializing the body, restoring parity with the DS path's load_imr_investor and removing the latent type-confusion exposure. A test also locks the hardcoded discriminator constant to the value Anchor derives.

Cyfrin: Verified.

7.1.3 `securitize_bridge::initialize` applies no validation to the SPL `acl_program_id`, permanently bricking an instance from a zero or garbage value

Description: `initialize` validates the `Ds` variant's `authorized_user_role` against the zero pubkey but applies NO validation to the `Spl` variant's two stored program ids. `token_config` is taken verbatim from caller instruction data and stored directly as `BridgeConfig::token_config`; for a `TokenConfig::Spl { acl_program_id, spl_token_registry_program_id }` the only structural check performed is `require_matches_mint`, which inspects the mint authority structure, not the two config program ids:

```
if let TokenConfig::Ds { authorized_user_role } = &token_config {
    require_keys_neq!(*authorized_user_role, ZERO_PUBKEY, BridgeError::RbacNotConfigured);
}
// no equivalent check for the Spl variant
```

Both SPL program ids can therefore be initialized to `Pubkey::default` or any garbage value. This is asymmetric with the `Ds` variant's own init check, with the consumer instructions `execute_vaa_v1_spl` and `bridge_spl_tokens` (whose account constraints reject a zero registry program id at runtime), and with the setter `update_spl_token_registry_program_id` (which rejects the zero pubkey). The `spl_token_registry_program_id` half is recoverable post-init via that setter, but there is no setter anywhere in the program for `acl_program_id` - the only writes to it are in `initialize` itself and in `update_spl_token_registry_program_id`, the latter preserving the existing `acl_program_id` unchanged. An `acl_program_id` written wrong at init is therefore an irreversible deploy action for that bridge instance.

With a wrong `acl_program_id` stored, every inbound mint reverts: `execute_vaa_v1_spl` calls `require_spl_matching` against the real ACL program account the relayer passes, which never equals the wrong stored value, yielding `InvalidAclProgram`. No instruction can rewrite `acl_program_id`, and because `BridgeConfig` is a per-mint PDA created with `init`, the operator cannot re-`initialize` over it (the second `init` fails - the PDA already exists). Recovery requires a program upgrade or operating a different asset mint. Only the program upgrade authority can call `initialize`, so this harm materializes only when that trusted operator mis-enters the value.

Files:

- `securitize_bridge::initialize` - `bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/admin/initialize.rs:118-133`

Impact: A fresh SPL bridge instance whose `acl_program_id` was mis-entered at deploy time is permanently non-functional for the inbound EVM-to-Solana mint path, with no on-chain recovery short of a program upgrade. No funds are lost - inbound simply reverts and the outbound burn path is independent of `acl_program_id` - so this is a deploy-time footgun and availability gap rather than a value loss. The `spl_token_registry_program_id` half of the same missing-validation gap is fully recoverable via its setter, so only `acl_program_id` carries the irreversibility.

Recommended Mitigation: In `initialize`, extend the `token_config` validation to the `Spl` branch symmetric with the existing `Ds` branch, rejecting a zero `acl_program_id` and a zero `spl_token_registry_program_id`:

```
if let TokenConfig::Spl { acl_program_id, spl_token_registry_program_id } = &token_config {
    require_keys_neq!(*acl_program_id, ZERO_PUBKEY, BridgeError::InvalidAclProgram);
    require_keys_neq!(*spl_token_registry_program_id, ZERO_PUBKEY,
    ↪  BridgeError::SplTokenRegistryProgramNotConfigured);
}
```

Separately, consider adding an `update_acl_program_id` setter (mirroring `update_spl_token_registry_program_id`) so that a mis-entered `acl_program_id` is recoverable without a program upgrade.

Securitize: Fixed in commit 13d365efc. initialize now validates the Spl branch symmetrically with the Ds branch — it rejects a zero acl_program_id (InvalidAclProgram) and a zero spl_token_registry_program_id (SplTokenRegistryProgramNotConfigured). Additionally, an update_acl_program_id setter was added (mirroring update_spl_token_registry_program_id) so a mis-entered acl_program_id is recoverable without a program upgrade, removing the irreversibility.

Cyfrin: Verified.

7.1.4 De-whitelisting closes `InvestorRegistry` but does not re-freeze the owner's token accounts

Description: The SPL whitelist program provides a single onboarding path (`whitelist`) that both provisions the per-`(mint, owner)` `InvestorRegistry` and thaws the owner's frozen Token-2022 accounts. The only reverse path exposed by the program is `delete_investor_registry`, which closes the registry PDA but leaves the owner's ATA(s) thawed. A wallet that has been removed from compliance metadata can therefore continue to send and receive tokens on Solana, even though cross-chain bridging is blocked by the missing registry.

Onboarding is fully handled inside `whitelist`:

```
/// Thaws each remaining account after validating ownership and mint/freeze authority,
/// then provisions the `(mint, owner)` `InvestorRegistry` if it does not exist yet.
pub fn whitelist_handler<'info>(
    mut ctx: Context<'_, '_, '_, 'info, Whitelist<'info>>,
    investor_id: String,
) -> Result<()> {
    // ...
    let registry_created = ensure_investor_registry(&mut ctx, &investor_id)?;
    // ...
    // Thaw the token accounts
    for token_account_info in token_accounts_infos.iter() {
        // ...
        require!(
            token_account.state
                == anchor_spl::token_2022::spl_token_2022::state::AccountState::Frozen,
            SplWhitelistErrorCode::AccountNotFrozen
        );

        freeze_authority_type.thaw_account(/* ... */)?;
    }
    // ...
}
```

The reverse operation only closes the registry account and refunds rent. It performs no token-account state change:

```
/// Closes the registry account and refunds rent to `receiver` (or `signer` when
/// `receiver` is omitted). Callable by admin or by a freeze authority of the mint.
pub fn delete_investor_registry_handler(ctx: Context<DeleteInvestorRegistry>) -> Result<()> {
    // ... authorization ...
    emit!(crate::events::InvestorRegistryDeleted {
        mint: ctx.accounts.investor_registry.mint,
```

```
        wallet: ctx.accounts.investor_registry.wallet,
    });

    let rent_receiver = ctx
        .accounts
        .receiver
        .as_ref()
        .map(|r| r.to_account_info())
        .unwrap_or_else(|| ctx.accounts.signer.to_account_info());

    ctx.accounts.investor_registry.close(rent_receiver)?;

    Ok(())
}
```

There is no symmetric `unwhitelist` / `delist` instruction in the program that accepts the owner's token accounts and CPIs into ACL/`token_2022::freeze_account`.

Impact: Operators may treat registry deletion as complete de-whitelisting, while cross-chain blocking and on-chain transfer blocking diverge. Cross-chain is disabled but on-chain transfers are not.

Recommended Mitigation: Add a symmetric offboarding path that mirrors `whitelist`.

Securitize: Acknowledged as by design. The program-level asymmetry doesn't cause the described divergence because freezing is intentionally decoupled from registry management and handled off-chain by our trusted back-end.

7.2 Informational

7.2.1 `securitize_bridge` inbound SPL resolver hardcodes the Token-2022 program for ATA derivation, breaking inbound delivery for a classic-SPL-Token mint configured as `TokenConfig::Spl`

Description: The executor resolver for the SPL inbound path unconditionally derives the recipient associated token account (and the `token_program` account meta) using the Token-2022 program id:

```
let token_program = anchor_spl::token_2022::ID;
let recipient_token_account = get_associated_token_address_with_program_id(
    destination_wallet, asset_mint, &token_program,
);
```

By contrast, the `execute_vaa_v1_spl` and `bridge_spl_tokens` handlers take `token_program: Interface<'info, TokenInterface>` and bind the ATA to `associated_token::token_program = token_program`, i.e. they accept whichever token program actually owns the mint. The `TokenConfig::Spl` variant is detected at `initialize` purely by the mint authority NOT being owned by the asset controller (the false branch of the DS-mint check); nothing pins an SPL-variant mint to the Token-2022 program specifically.

If an operator configures an `Spl` bridge for a mint owned by the classic SPL Token program, every executor-driven inbound relay derives the recipient ATA under Token-2022 - a different address than the real classic-SPL ATA the mint expects. When the resolved account list is fed into `execute_vaa_v1_spl`, the `asset_mint` (`InterfaceAccount<Mint>`) deserialize requires the mint's owner to equal the supplied `token_program` (Token-2022), but the mint is owned by classic SPL Token, so account resolution fails and the inbound path for that mint cannot execute via the standard executor flow. The fault manifests under honest configuration - no malicious actor is required - but only for an SPL-configured mint that is classic SPL Token rather than Token-2022.

Files:

- `securitize_bridge::derive_execute_vaa_accounts_spl` - `bc-solana-bridge-sc/programs/securitize_bridge/src/resolver/spl.rs:126-132`

Impact: The standard executor-driven inbound path is broken for any `Spl`-configured mint that is a classic SPL Token rather than Token-2022: the resolver builds an account list whose recipient ATA and `token_program` meta are derived under Token-2022, which `execute_vaa_v1_spl` then rejects when it deserializes the classic-SPL `asset_mint`. The direct `execute_vaa_v1_spl` instruction itself is not inherently broken - its handler binds the ATA to the caller-supplied `token_program`, so a manual submission with the correct classic-SPL token program and matching ATA can still execute - but the automated executor relay (which uses this resolver) cannot deliver. No value is lost: the VAA is simply never consumed on the failed path (the `consumed_vaa` replay PDA is created with `init` and is only written on a successful instruction), so the burned source tokens remain redeemable via a correctly-constructed manual `execute_vaa_v1_spl` and the failure fails closed. Because the SPL product is documented as Token-2022-only, the realistic blast radius is a misconfiguration-class denial of the automated inbound path for the affected instance, with no value loss.

Recommended Mitigation: Derive the recipient ATA's token program from the live mint account's owner rather than hardcoding the Token-2022 program. In the resolver, read the `asset_mint` account's owner and pass that program id into `get_associated_token_address_with_program_id` and the `token_program` account meta, so the resolved ATA matches the address the handler derives. Alternatively, pin the SPL variant to Token-2022 explicitly at `initialize` (assert the mint account owner equals the Token-2022 program for `TokenConfig::Spl`) so the resolver hardcode and the config are provably consistent.

Securitize: Fixed in commit cd5026d. Fixed via the second recommended option, aligned with the documented Token-2022-only design. initialize now pins the SPL variant to Token-2022: for TokenConfig::Spl it asserts the asset_mint account owner equals the Token-2022 program (SplMintNotToken2022), so the resolver's hardcoded Token-2022 ATA derivation and the stored config are provably consistent. This makes the misconfiguration fail-fast at deploy time — an operator can no longer create an SPL instance over a classic-SPL mint — rather than silently breaking the automated inbound path later.

Cyfrin: Verified.

7.2.2 Inconsistent `investor_id` length limits across layers (64 vs 256) and an undocumented 32-byte PDA-seed constraint on the DS path

Description: The `investor_id` field has several different maximum-length limits scattered across the codebase (64 and 256), defined under different constant names and enforced at different layers. None of these is a security flaw on its own, but the divergence is a source of confusion and there is an additional, implicit 32-byte limit on the DS path (PDA seed) that is neither documented nor explicitly validated against the 256-byte wire limit.

`investor_id` is not defined in a single place. It originates from two different external sources depending on the bridge flow, and is bounded by different constants at each layer:

Limit	Constant	Location
256	`MAX_INVESTOR_ID_LEN`	`bc-solana-bridge-sc/programs/securitize_bridge/src/utils/payload.rs:24`
64	`INVESTOR_ID_MAX_LEN`	`bc-solana-bridge-sc/programs/securitize_bridge/src/state/investor_regi`
64	`INVESTOR_ID_MAX_LEN`	`bc-solana-whitelist-sc/programs/spl-token-whitelist/src/constants.rs:1`
32 (implicit)	— (Solana `MAX_SEED_LEN`)	`bc-solana-bridge-sc/programs/securitize_bridge/src/resolver/ds.rs:41-4`

Impact: The divergent limits create maintainability/clarity risk: a reader cannot tell from any single constant what the real bound on `investor_id` is.

Recommended Mitigation: Consolidate the `investor_id` length limits behind a single, clearly named, shared constant

Securitize: Fixed in commits 6fef9983 and 26352c4. We verified that the canonical investor_id is always 32 bytes, so we consolidated all investor_id length limits to a single value of 32 rather than just renaming: the bridge uses one shared INVESTOR_ID_MAX_LEN = 32 for both the DS-seed check and the SPL InvestorRegistry read-view, and the whitelist storage cap was lowered from 64 to 32. The cross-chain ABI wire bound stays a separate, clearly-named MAX_ENCODED_INVESTOR_ID_LEN = 256 (a deliberately looser envelope). The previously-implicit 32-byte DS PDA-seed limit is now an explicit, named constant with an upfront require!.

Cyfrin: Verified.

7.2.3 Missing upfront validation during initialization

Description: During bridge initialization, SPL token configuration is accepted without fully validating that the SPL mint is controlled by the expected ACL PDA, and without checking that `spl_token_registry_program_id` is non-zero. The initialization flow only checks that the token config broadly matches the mint type:

```
token_config.require_matches_mint(
    &ctx.accounts.asset_mint.to_account_info(),
    &ctx.accounts.mint_authority.to_account_info(),
)?;
```

For SPL tokens, this only confirms the mint is not detected as a DS mint:

```
match (self, is_ds_mint) {
    (TokenConfig::Ds { .. }, true) => Ok(()),
    (TokenConfig::Spl { .. }, false) => Ok(()),
    _ => err!(BridgeError::InitParamsMismatch),
}
```

It does not validate that:

```
acl_program_id != ZERO_PUBKEY
spl_token_registry_program_id != ZERO_PUBKEY
mint_authority == expected ACL PDA
```

A similar non-zero check exists later when updating the registry program id correctly:

```
require!(
    new_spl_token_registry_program_id != ZERO_PUBKEY,
    BridgeError::SplTokenRegistryProgramNotConfigured,
);
```

But the same validation is not enforced during initialization.

Impact: This can allow an SPL bridge instance to be initialized with incomplete or incorrect ACL/registry configuration. The issue is mostly defense-in-depth because later bridge/execute paths may fail when they try to use the bad config, but the bridge can still be deployed into a broken state. This may cause outbound or inbound bridge operations to fail unexpectedly, especially inbound minting, where the ACL program and expected mint authority are required.

Recommended Mitigation: Add strict SPL config validation during initialization as well.

Securitize: Partially already fixed, remainder is intended design. The two non-zero checks (acl_program_id != 0, spl_token_registry_program_id != 0) are already enforced at initialize — added in audit fix commit 13d365efc (the issue text predates that change). The remaining point (mint_authority == expected ACL PDA at init) we keep as-is by design: ACL-PDA correctness is validated lazily at the first ACL CPI, initialize is permissioned to the program upgrade authority (so only the trusted deployer can misconfigure, no third-party exploit), and an upfront check would re-introduce a find_program_address deliberately removed for gas in audit v1.0 issue 17. No further change.

Cyfrin: Partial Fix verified. Remainder are marked as acknowledged as by design.

7.2.4 Outbound DS/SPL bridge handlers do not validate EVM recipient address format

Description: The DS and SPL outbound bridge instructions (`bridge_ds_tokens` and `bridge_spl_tokens`) accept any non-zero 32-byte `recipient`, without verifying that it is a well-formed EVM address for the target chain.

Both outbound DS and SPL handlers delegate recipient validation to the shared helper `check_bridge_send_args`, which only rejects the zero address and positive-amount checks:

```
pub fn check_bridge_send_args(
    asset_mint: &Pubkey,
    target_chain: u16,
    recipient: &[u8; 32],
    amount: u64,
) -> Result<()> {
    require_keys_neq!(*asset_mint, ZERO_PUBKEY, BridgeError::ZeroAssetMint);

    #[cfg(not(feature = "devnet"))]
    require!(
        target_chain != wormhole::CHAIN_ID_SOLANA,
        BridgeError::CannotBridgeToSameChain,
    );
    #[cfg(feature = "devnet")]
    let _ = target_chain;

    require!(*recipient != ZERO_ADDRESS, BridgeError::InvalidRecipient);

    require_gt!(amount, 0, BridgeError::InvalidBridgeAmount);

    Ok(())
}
```

`bridge_ds_tokens` and `bridge_spl_tokens` both call this helper before burning/revoking tokens and posting the Wormhole message:

```
    check_bridge_send_args(&config.asset_mint, target_chain, &recipient, amount)?;
```

```
    check_bridge_send_args(&config.asset_mint, target_chain, &recipient, amount)?;
```

The encoded payload stores the caller-supplied `recipient` verbatim as `destination_wallet`:

```
    let payload_bytes = abi_encode_bridge_payload(&BridgePayload {
        target_chain,
        investor_id: investor_id.clone(),
        value: amount,
        investor_wallet: ctx.accounts.payer.key().to_bytes(),
        destination_wallet: recipient,
        country: String::new(),
        attribute_values: vec![0u64; 4],
        attribute_expirations: vec![0i64; 4],
    })?;
```

By contrast, the USDC bridge outbound handler applies an additional EVM-format check on the same 32-byte field:

```
    require_gt!(amount, 0, UsdcBridgeError::InvalidAmount);
    require!(recipient != ZERO_ADDRESS, UsdcBridgeError::InvalidRecipient);

    // A valid EVM address in 32-byte form must be left-padded with 12 zero bytes
    require!(
        recipient[..12] == [0u8; 12],
        UsdcBridgeError::InvalidEvmAddress,
    );
```

Impact: A user (or integrator) may pass a raw Solana pubkey, an unpadded hex string, or another malformed 32-byte value as the destination.

Recommended Mitigation: Align DS/SPL outbound validation with the USDC bridge.

Securitize: Fixed in commit d7e9ba1f128. The shared check_bridge_send_args helper (used by both bridge_ds_tokens and bridge_spl_tokens) now enforces the EVM-address format on the 32-byte recipient, aligning DS/SPL outbound with the USDC bridge: it requires the high 12 bytes to be zero (recipient[..12] == [0u8; 12], else InvalidEvmAddress).

Cyfrin: Verified.

7.2.5 Empty `investor_id` can be persisted in `InvestorRegistry` and propagated through the SPL bridge

Description: The SPL whitelist program accepts an empty string as a valid `investor_id` when creating an `InvestorRegistry` PDA. No minimum-length or non-empty check exists on either the admin/freeze-authority `create_investor_registry` path or the user `whitelist` path. Once written, the value is immutable unless the PDA is explicitly deleted. The bridge program reads this field as the compliance source of truth for outbound SPL transfers and embeds it verbatim in the Wormhole payload, so an empty registry entry yields cross-chain messages with no investor attribution.

`InvestorRegistry::new` is the sole validation gate for the stored identifier. It only rejects strings longer than 64 bytes; an empty string (`len == 0`) passes:

```
    pub fn new(mint: Pubkey, wallet: Pubkey, investor_id: &str, bump: u8) -> Result<Self> {
        require!(
            investor_id.len() <= INVESTOR_ID_MAX_LEN,
            SplWhitelistErrorCode::InvestorIdTooLong
        );
```

Impact: Allowing an empty value undermines that invariant: whitelisted wallets can move tokens cross-chain without attaching any investor identifier to the wire payload.

Recommended Mitigation: Add an explicit non-empty validation.

Securitize: Fixed in commits 91a4db6 and f3f7d146. The whitelist program's InvestorRegistry::new — the single chokepoint for both the admin/freeze-authority create_investor_registry path and the user whitelist path — now rejects an empty or whitespace-only investor_id (InvestorIdEmpty), so no empty record can be persisted. As defense-in-depth (the bridge and whitelist deploy independently), the bridge's outbound payload encoder (validate_encode_bounds) also rejects an empty/whitespace investor_id before embedding it in the Wormhole payload.

Cyfrin: Verified.

7.2.6 `tracker_account` binding is not validated upfront in `bridge_ds_tokens`

Description: In `bridge_ds_tokens`, `tracker_account` is accepted as an unchecked, caller-supplied account and is read by the bridge's own lock-up gate (`validate_locked_tokens`) before any binding to the current `(asset_mint, identity_account)` pair is verified. The only effective binding enforcement happens several CPIs later inside the Policy Engine (`update_counters_on_burn`). For defense in depth, the bridge should validate `tracker_account` at the front, before relying on its contents.

`tracker_account` is declared as an `UncheckedAccount` with no PDA seeds and no relationship to `identity_account` or `asset_mint`:

```
    /// CHECK: Passed to RBAC CPI and validate_locked_tokens.
    #[account(mut)]
    pub tracker_account: UncheckedAccount<'info>,
```

Impact: If a mismatched `tracker_account` is supplied, the Policy Engine CPI eventually reverts. However, correctness currently depends on an unrelated downstream program several CPIs away.

Recommended Mitigation: The project should address this in one of two ways:

- either fix it in code
- or make the deferred-validation design explicit in documentation.

Securitize: Fixed in commit 318bdd5. validate_locked_tokens now validates the tracker_account binding upfront, before relying on its contents: it asserts tracker.asset_mint equals the bridge's asset_mint and tracker.identity_account equals the supplied identity_account, returning a new TrackerAccountMismatch error on mismatch. The bridge no longer depends on the downstream Policy Engine CPI to catch a mismatched tracker — the binding is enforced at the front of bridge_ds_tokens.

Cyfrin: Verified.

7.2.7 Whitelist pause does not stop investor registry create/delete paths

Description: The main `whitelist` instruction checks the global pause flag:

```
require!(
    !ctx.accounts.spl_whitelist_state.is_paused,
    SplWhitelistErrorCode::Paused
);
```

However, the direct registry mutation paths do not check `is_paused`. So, while paused, the combined whitelist/thaw flow is blocked, but authorized callers can still create or delete InvestorRegistry records directly.

Impact: If pause is intended as an emergency stop for all whitelist-related state changes, this creates a bypass. During a pause, admin or freeze-authority accounts can still alter which wallets are considered registered for SPL bridge usage.

Recommended Mitigation: Add the same pause guard to create_investor_registry_handler and delete_investor_registry_handler.

Securitize: Acknowledged — keeping as-is by design. create_investor_registry and delete_investor_registry are privileged admin/freeze-authority instructions, not permissionless user paths.

Cyfrin: The team acknowledged as by design since these are privileged instructions.

7.3 Gas Optimization

7.3.1 Redundant heap clone of the up-to-1024-byte VAA payload on every inbound bridge transaction

Description: `validate_and_decode_vaa` deliberately `core::mem::take`s the raw payload bytes out of `PostedVaaData` into `DecodedVaa::posted_payload_bytes` "to avoid an extra heap copy" - but `finalize_inbound_vaa` then immediately `.clone()`s that buffer back into `received.payload`. Because `DecodedVaa` is passed to `finalize_inbound_vaa` by shared reference (`&decoded`) and the `posted_payload_bytes` field is never read again by either inbound handler after this call (both `execute_vaa_v1` and `execute_vaa_v1_spl` only read `decoded.payload.*` / `decoded.posted_emitter_chain` afterwards), the clone is pure waste: a heap allocation plus a memcpy of up to `PAYLOAD_MAX_LENGTH` (1024) bytes on every inbound bridge execution (a hot, per-transaction path).

```
bc-solana-bridge-sc/programs/securitize_bridge/src/utils/validate_and_decode_vaa.rs
76:         posted_payload_bytes: core::mem::take(&mut posted_vaa.payload),
97:     received.payload = decoded.posted_payload_bytes.clone();
```

Call sites confirming the field is not reused after `finalize_inbound_vaa`:

```
bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/bridge/execute_vaa_v1.rs
232:     finalize_inbound_vaa(
257:     emit_cpi!(DsTokenBridgeReceive { ... decoded.payload.* ... })

bc-solana-bridge-sc/programs/securitize_bridge/src/instructions/bridge/execute_vaa_v1_spl.rs
177:     finalize_inbound_vaa(
187:     emit_cpi!(SplTokenBridgeReceive { ... decoded.payload.* ... })
```

Recommended Mitigation: Move the buffer instead of cloning it. Change `finalize_inbound_vaa` to take the payload bytes by value (or to take `&mut DecodedVaa` and `core::mem::take` the field), so the already-owned `Vec<u8>` is moved straight into `received.payload`:

```rust
pub fn finalize_inbound_vaa(
    consumed_vaa: &mut Account<ConsumedVaa>,
    received: &mut Account<Received>,
    recipient_wallet: &Pubkey,
    decoded: &mut DecodedVaa,   // was &DecodedVaa
    vaa_hash: [u8; 32],
) -> Result<()> {
    // ... existing checks (read decoded.destination_wallet etc.) ...
    received.payload = core::mem::take(&mut decoded.posted_payload_bytes); // no clone
    Ok(())
}
```

The `take` already on line 76 plus the move here makes the whole inbound path move-only for the payload buffer. The `emit_cpi!` blocks that follow do not touch `posted_payload_bytes`, so emptying it is safe.

Securitize: Fixed in commit 5b4982a1bb. finalize_inbound_vaa now takes decoded: &mut DecodedVaa and moves the payload buffer into received.payload via core::mem::take(&mut decoded.posted_payload_bytes) instead of cloning it. Combined with the existing take out of PostedVaaData, the whole inbound path is now move-only for the payload buffer — the up-to-1024-byte heap allocation with memcpy per inbound transaction is eliminated. Behavior is unchanged.

Cyfrin: Verified.